

Hackathon Problem Statement

Author: j.balakrishnan

The solution to be called as “Voice Agentic AI Assistant “, should demonstrate strong Python programming skills, AI integration capability, data handling, and communication workflow automation.

Instructions to Candidates:

1. Below document is reviewed and baselined – there will NOT be any clarification session/communication between the author of the document or the team associate.
2. There will NOT any software, API Keys, additional documentation, additional technical assistance provided
3. Candidate need to submit the final outcome including the code, demos to the supervisor for further evaluation – either it can be a shared Github location or a designed work location shared by the supervisor
4. Evaluation of the work completed by the candidate is based on the matrix as defined at the end of this document.
5. Follow the instruction as mentioned by the supervisor for time completion, break between session and other security standards to be followed at discretion of the company policies

Objective:

Build an intelligent Voice Agentic AI Assistant using Python that can:

1. Accept voice input from users
2. Convert voice to text
3. Process the text using RegEx, pattern matching, and GenAI capabilities
4. Generate intelligent responses using Gemini AI or any open-source LLM
5. Convert AI-generated text response back to voice
6. Store all conversations/responses in CSV files for tracking and analytics

Suggested Technology Stack:

Area	Suggested Technologies
Programming Language	Python
Voice-to-Text	SpeechRecognition, Whisper
Text-to-Voice	pyttsx3, gTTS
GenAI	Gemini AI, Ollama, Mistral, Llama OR any other opensource
Data Storage	CSV
RegEx	Python re library

UI (Optional)	Streamlit, Flask
Utilities	Pandas, Collections, Sorting libraries

Functional Requirements:

Here are high level functional requirements expected from the candidates

1. Voice-to-Text

- a. Capture voice input through microphone/audio file
- b. Convert speech into text using Python libraries/APIs

2. Text Processing

- a. Use extensive **RegEx and pattern matching**
- b. Identify:
 - i. Greetings
 - ii. Dates
 - iii. Email IDs
 - iv. Phone numbers
 - v. Keywords/intents
 - vi. Commands
- c. Use Python collections, sorting, filtering, searching, etc.

3. GenAI / LLM Integration

- a. Integrate: with Google Gemini AI OR Open-source LLM (Mistral, Llama, Ollama, etc.)
- b. Generate meaningful responses based on user queries

4. Text-to-Voice: Convert generated text response back to speech

5. File Storage

- a. Store the following either in XLSX/CSV:
 - i. User voice transcript
 - ii. AI response
 - iii. Timestamp
 - iv. Intent/category
- b. Use CSV file-based persistence

6. Basic Agentic Behavior

The system should:

- a. Understand context
- c. Perform conditional actions
- d. Route requests based on detected intent/patterns

Example:

- a. "Show today's tasks"
- b. "Save this note"
- c. "Summarize this message"

Evaluation criteria:

Please note the following:

- a. Final scoring and review comments will be retained by the supervisor and will not be shared to the candidates
- b. Candidates who scored with a satisfactory scoring pattern will be processed for next level discussion
- c. Below is general guideline for evaluation and final scoring of the submitted work will be decided by the supervisor/designed authority and NO contest on final scoring encouraged

Evaluation Area	Criteria	Marks
Python Programming Skills	Clean coding, modularity, library usage, sorting/filtering logic	15
RegEx & Pattern Matching	Effective extraction, validation, intent detection	20
Voice-to-Text Implementation	Accuracy and usability	10
Text-to-Voice Implementation	Response quality and integration	10
GenAI / LLM Integration	Proper AI usage and meaningful responses	20
CSV/File Handling	Proper storage and retrieval	5
Agentic Workflow Capability	Conditional flows, action handling	10
Error Handling & Edge Cases	Stability and validation	5
Documentation & Demo	Clarity of explanation and presentation	5